Software Architecture

Minsoo Ryu

Hanyang University

Contents

1. Definitions
2. View Model
3. Architecture Patterns
4. Architecture Documentation

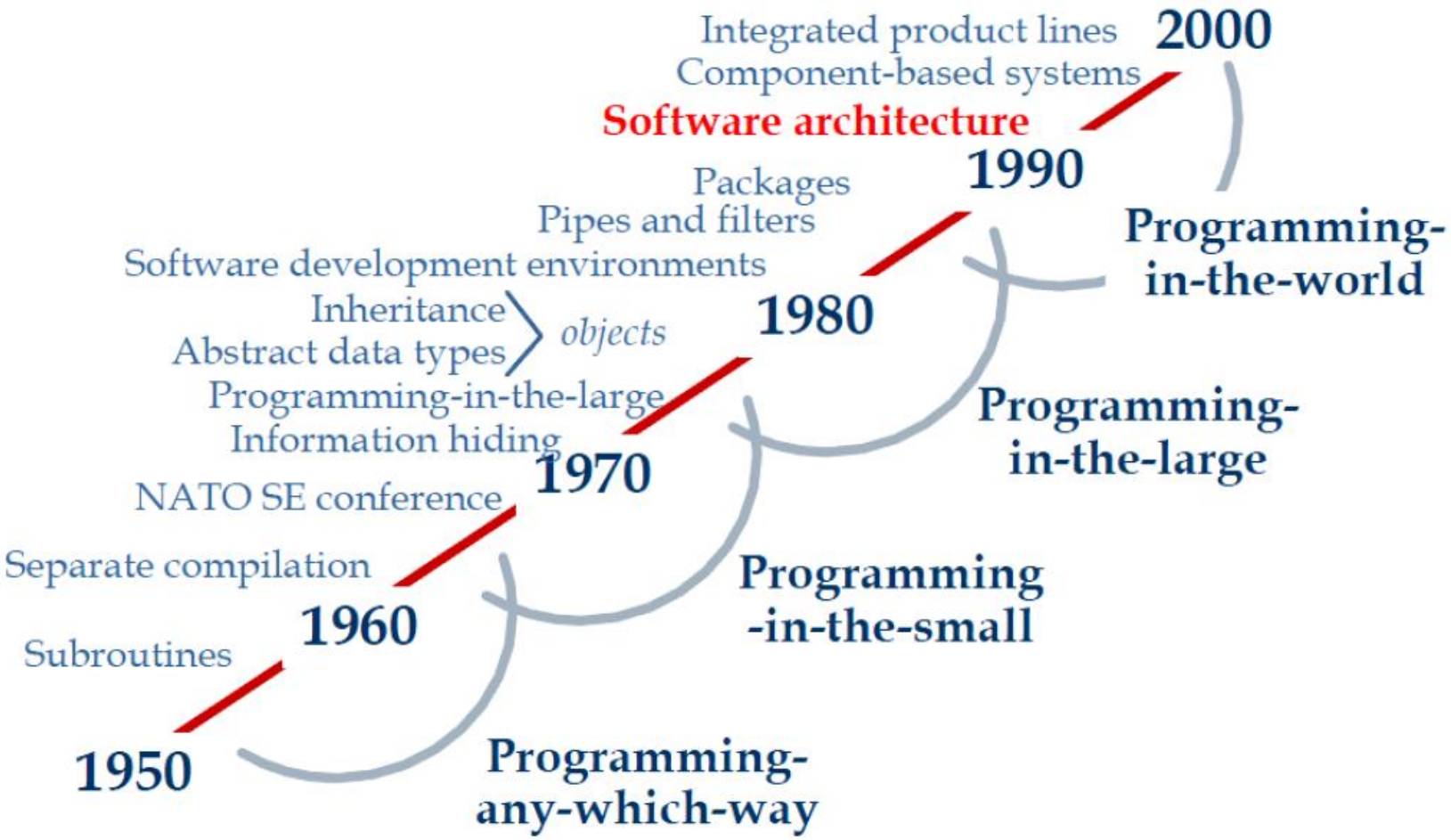
Software Architecture

- "The architecture of a system is a comprehensive framework that describes its form and structure—its components and how they fit together," by Jerrold Grochow

- To date there is still no agreement on the precise definition of the term "software architecture"

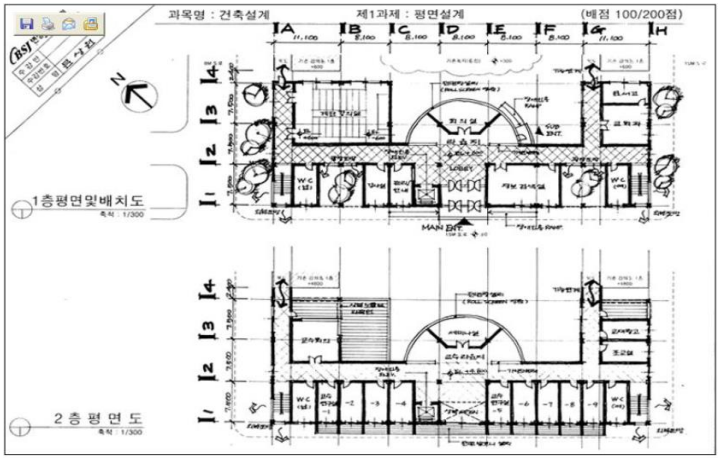
- Architectural design is the initial design process of identifying these sub-systems and establish a structural framework for sub-system control and communications

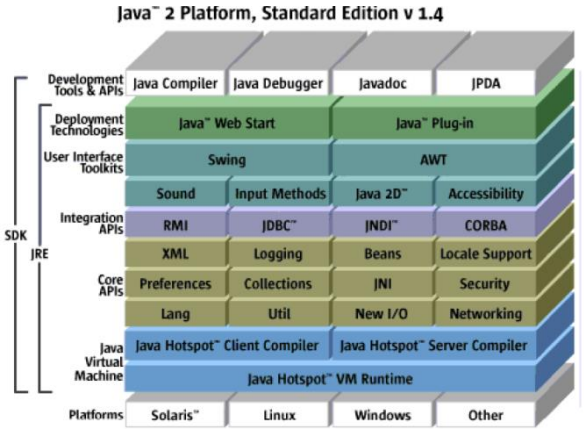
History


Integrated product lines
2000
Component-based systems
Software architecture
1990
Packages
Pipes and filters
Programming-
in-the-world
Software development environments
Inheritance
objects
Abstract data types
1980
Programming-in-the-large
Information hiding
Programming-
in-the-large
1970
NATO SE conference
Separate compilation
Programming
-in-the-small
1960
Subroutines
1950
Programming-
any-which-way
@2004, Antony J. Lattanze and David Garlan

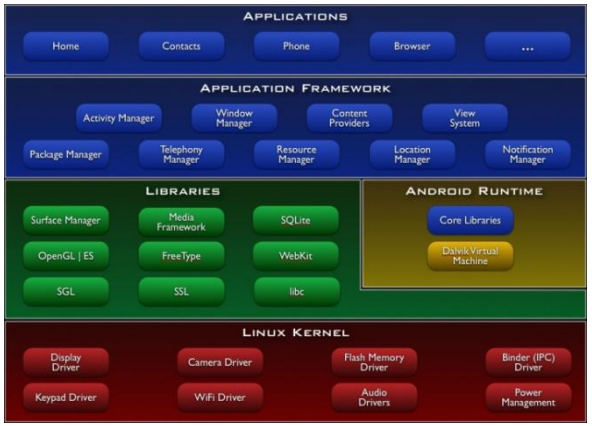
Why Is Software Architecture Important?











Why is Software Architecture Important?

- **Early design decisions**
  - Software architecture manifests the earliest design decisions governing the system with respect to the system's remaining development, its deployment, and its maintenance life
- **Communication among stakeholders**
  - Software architecture represents a common abstraction of a system that the system's stakeholders can use as a basis for mutual understanding, negotiation, consensus, and communication
- **Transferable abstraction of a system**
  - It can be applied to other systems exhibiting similar quality attribute and functional requirements and can promote large-scale re-use

Roles and Impacts

- Architecture serves as the blueprint for the system but also the project:
  - Team structure
  - Documentation organization
  - Work breakdown structure
  - Scheduling, planning, budgeting
  - Unit testing, integration

What is Software Architecture?

- **IEEE 1471-2000**
  - **Software architecture is the fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution**

- **Perry and Wolf, 1992**
  - **A set of architectural (or design) elements that have a particular form**
  - **Architecture = {Elements, Form, Rationale}**
    - Elements: components, connectors
    - Form: structure of components and connectors
    - Rationale: reasons of selecting elements and connectors

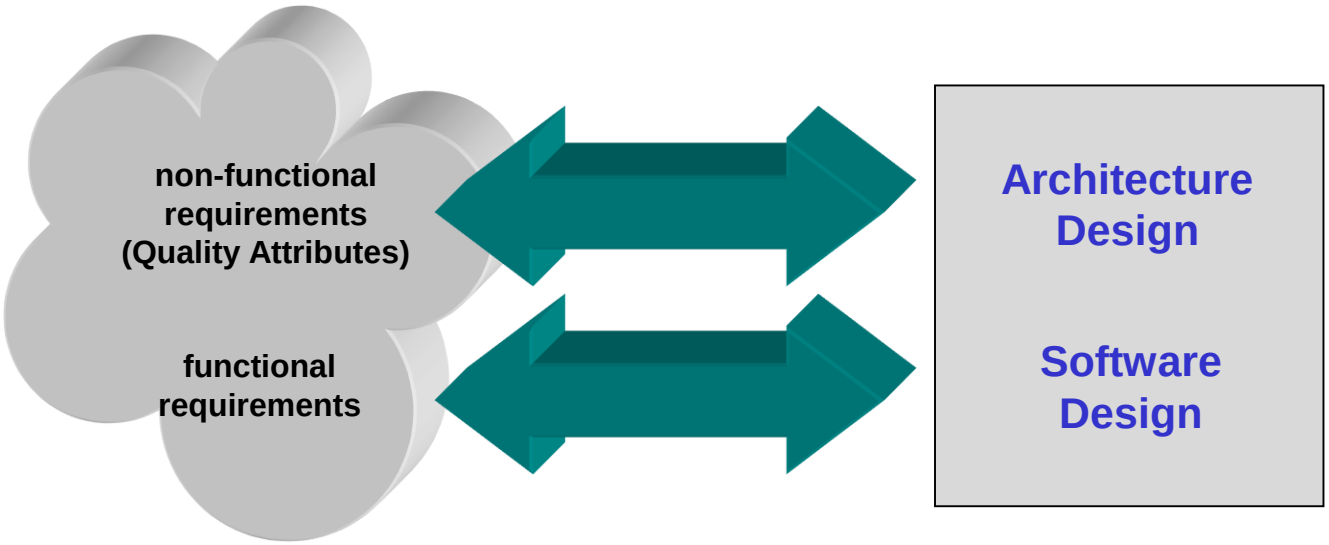
What is Software Architecture?

- **Boehm et al., 1995**
  - A collection of software and system components, connections, and constraints
  - A collection of system stakeholders' need statements
  - A rationale which demonstrates that the components, connections, and constraints define a system that, if implemented, would satisfy the collection of system stakeholders' need statements

- **Clements et al., 1997**
  - The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them

Architecture Design vs. SW Design

- **Architecture design**
  - **Non-functional decisions are cast, and functional requirements are partitioned**
- **Software design**
  - **Functional requirements are accomplished**



Software Quality Attributes

- Performance
- Availability
- Usability
- Security

End User's View

- Maintainability
- Portability
- Reusability
- Testability

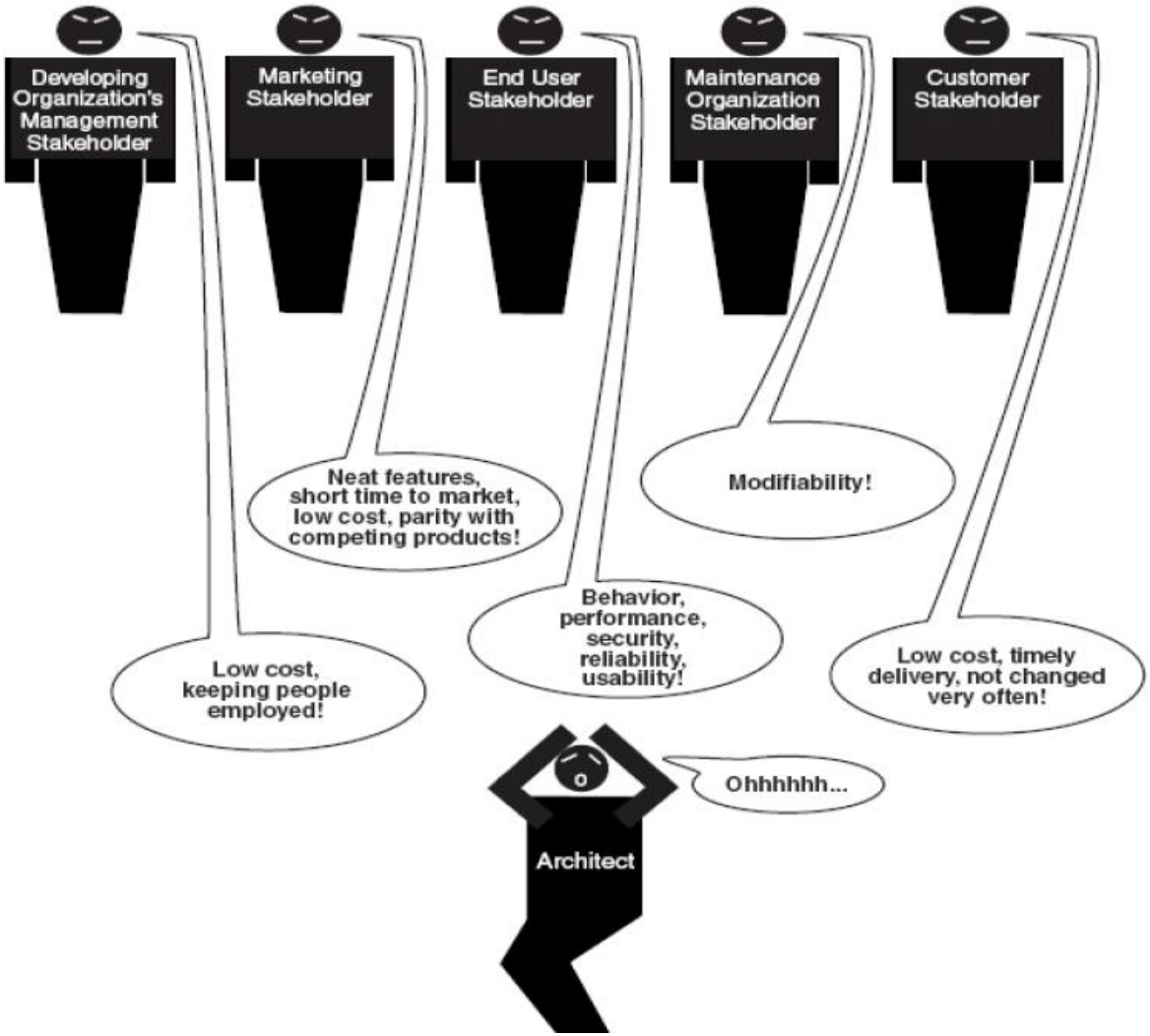
Developer's View

- Time To Market
- Cost and Benefits
- Projected life time
- Targeted Market
- Integration with Legacy System
- Roll back Schedule

Business Community View

ISO/IEC 9126-2001 Information Technology – Software Product Quality

Stakeholders



Contents

1. Definitions
2. View Model
3. Architecture Patterns
4. Architecture Documentation

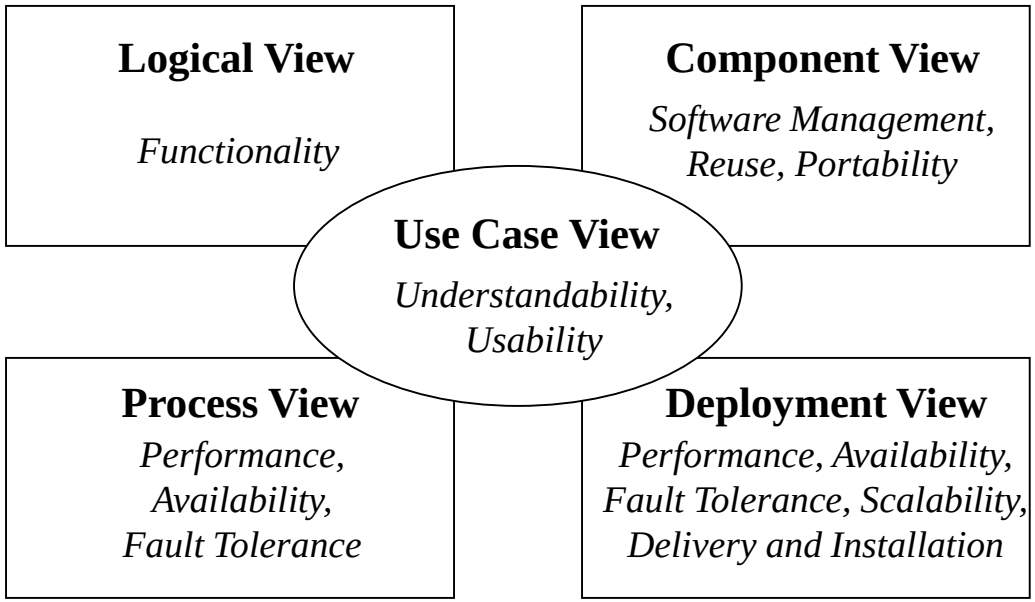
View Model

- A view model or viewpoints framework in systems engineering and software engineering is a framework which defines a coherent set of views to be used in the construction of a system architecture and software architecture

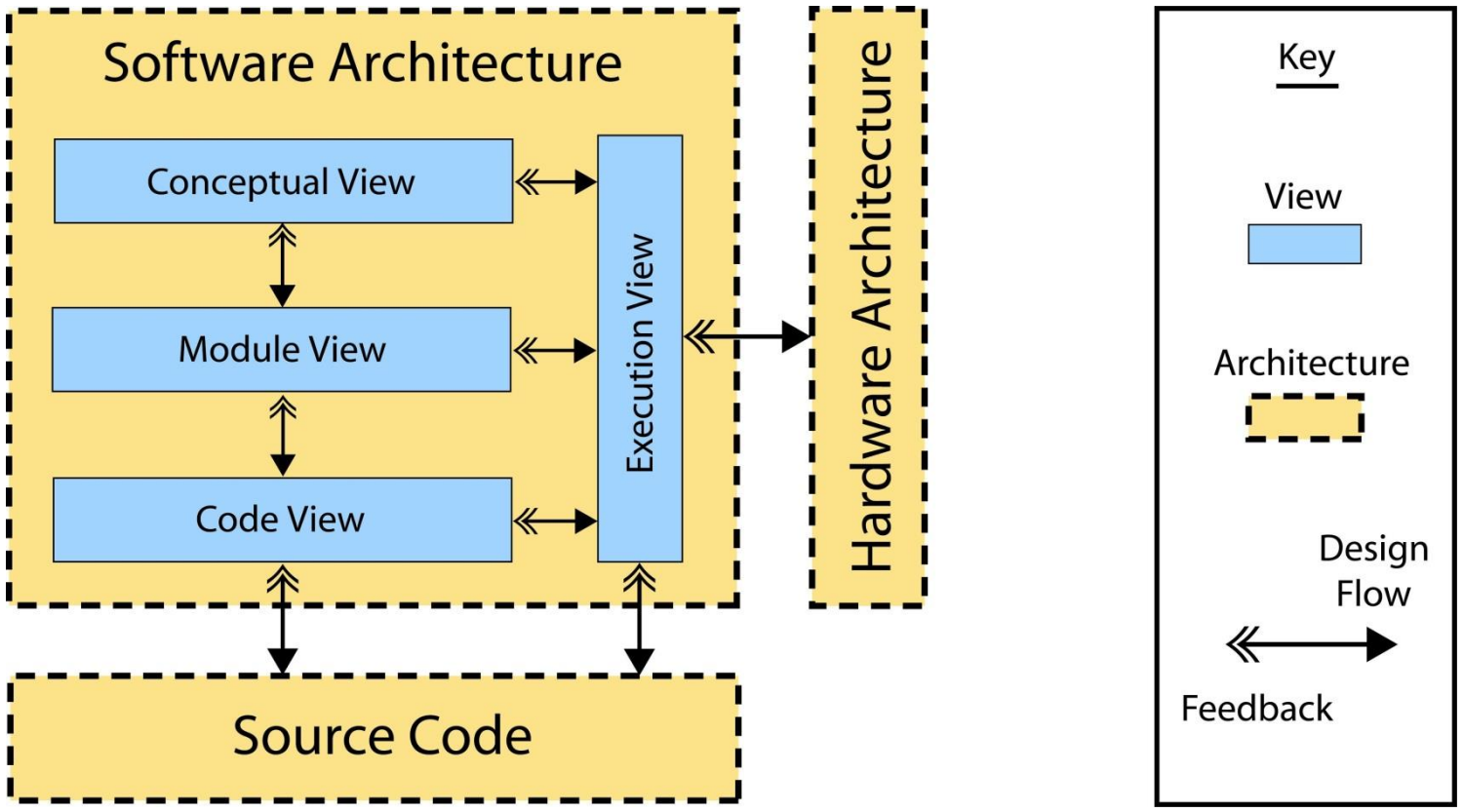
- A view is a representation of a whole system from the perspective of a related set of concerns
  - A viewpoint identifies the set of concerns and the representations/modeling techniques, etc. used to describe the architecture to address those concerns (IEEE 1471)
  - A view is the result of applying a viewpoint to a particular system (IEEE 1471)

“4+1” Software Architecture (Philippe Kruchten,1997)

- Logical view
  - class diagram level
- Component view
  - source code level
  - one-to-one mapping between classes and components
- Process view
  - execution level
- Deployment view
  - the configuration of processing elements and the software processes



Siemens Four View Model

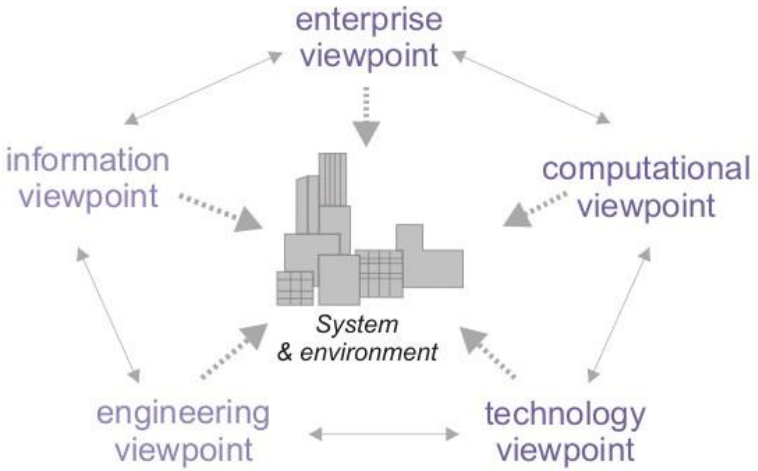


Siemens Four View Model

- **Conceptual view activities**
  - Create conceptual elements such as components, connectors, ports, roles, protocols, and configuration
- **Module view activities**
  - Define subsystems, layers, modules
  - Allocate conceptual elements to the above
- **Execution view activities**
  - Define run-time entities, communication paths, execution configuration

RM-ODP Views

- **ISO Reference Model for Open Distributed Processing**
  - **Specifies a set of viewpoints for partitioning the design of a distributed software/hardware system**
- **Five viewpoints**
  - **Enterprise viewpoint**
    - business objective/processes
  - **Information viewpoint**
  - **Computational viewpoint**
    - Computational components
  - **Engineering viewpoint**
    - Mechanisms and functions to support the interactions of computational components
  - **Technology viewpoint (implementation)**



Contents

1. Definitions
2. View Model
3. Architecture Patterns
4. Architecture Documentation

Architectural Patterns (or Styles)

- The architectural model of a system may conform to a generic architectural model or style
  - An awareness of these styles can simplify the problem of defining system architectures
  - However, most large systems are heterogeneous and do not follow a single architectural pattern

- Each style describes a system category that encompasses:
  - A set of components (e.g., a database, computational modules) that perform a function required by a system
  - A set of connectors that enable "communication, coordination and cooperation" among components
  - Constraints that define how components can be integrated to form the system
  - Semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts

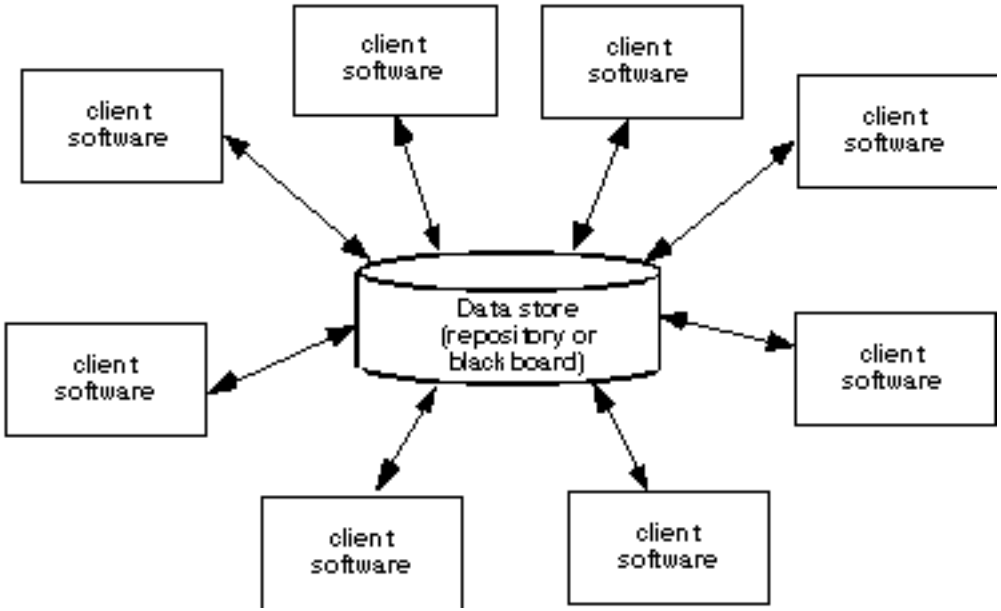
A Brief Taxonomy of Architectural Styles

- Although millions of computer-based systems have been created over the past 50 years, the vast majority can be categorized into one or a relatively small number of architectural styles

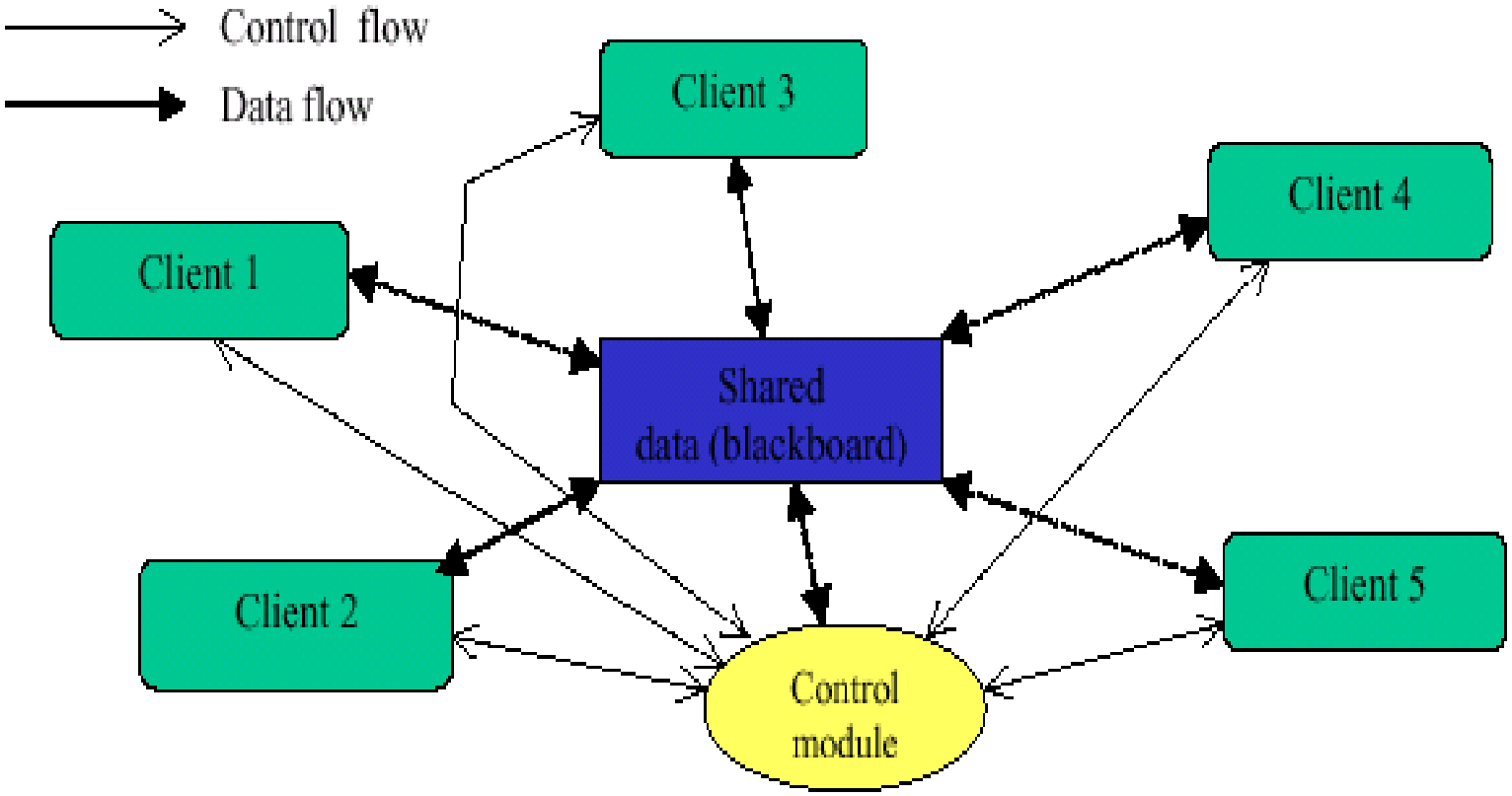
- Taxonomy
  - Data-centered architecture
  - Data-flow architecture
  - Layered architecture
  - Model-View-Controller architecture
  - Automatic control systems architecture
  - Microservice architecture
  - Remote Procedure Call (RPC)
  - Call and return architecture

Data-Centered Architecture

- Also called repository architecture
- A data store (a file or database) resides at the center of this architecture and is accessed frequently by other components



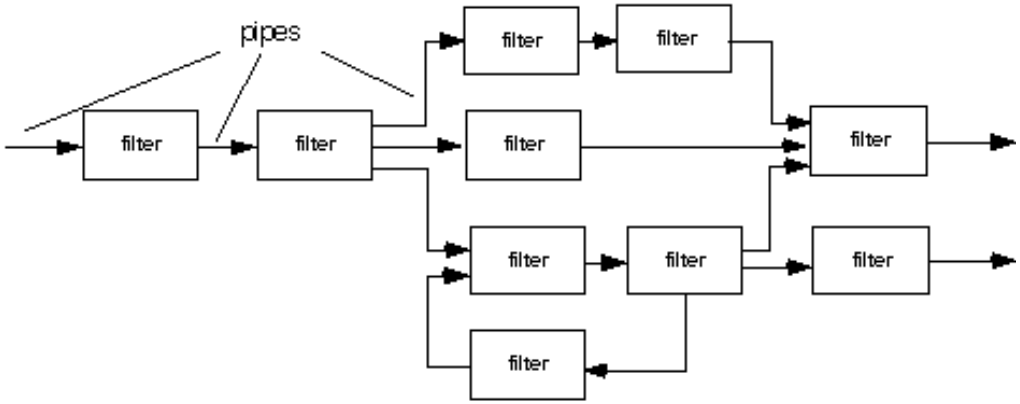
Blackboard (Data-Centered)



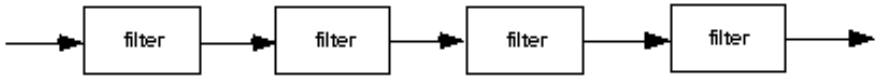
The Blackboard architectural style.

Data-Flow Architecture

- Input data are transformed through a series of computational or manipulative components into output data



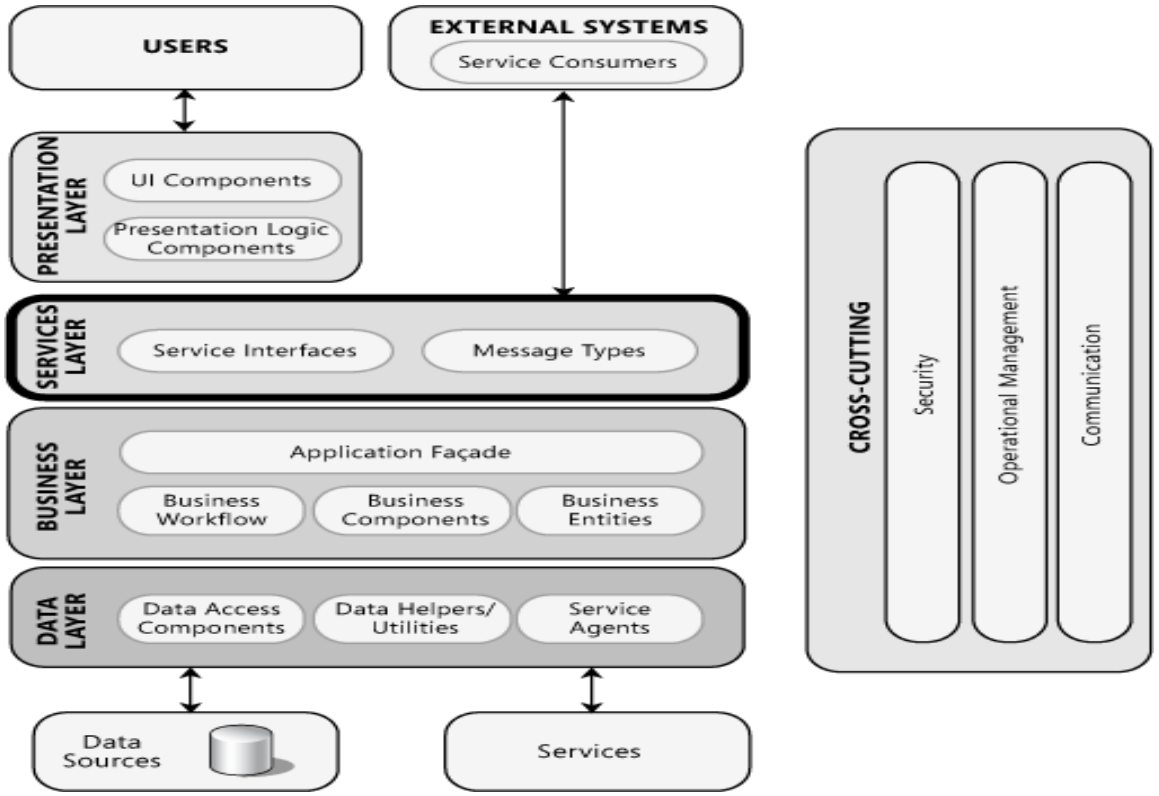
(a) pipes and filters



(b) batch sequential

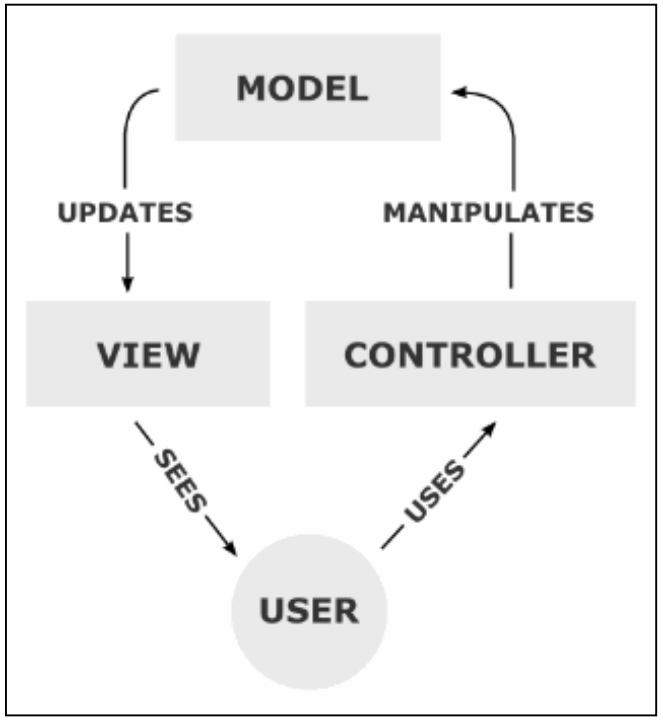
Layered Architecture

- Each layer (strictly) depends on only the adjacent layer below

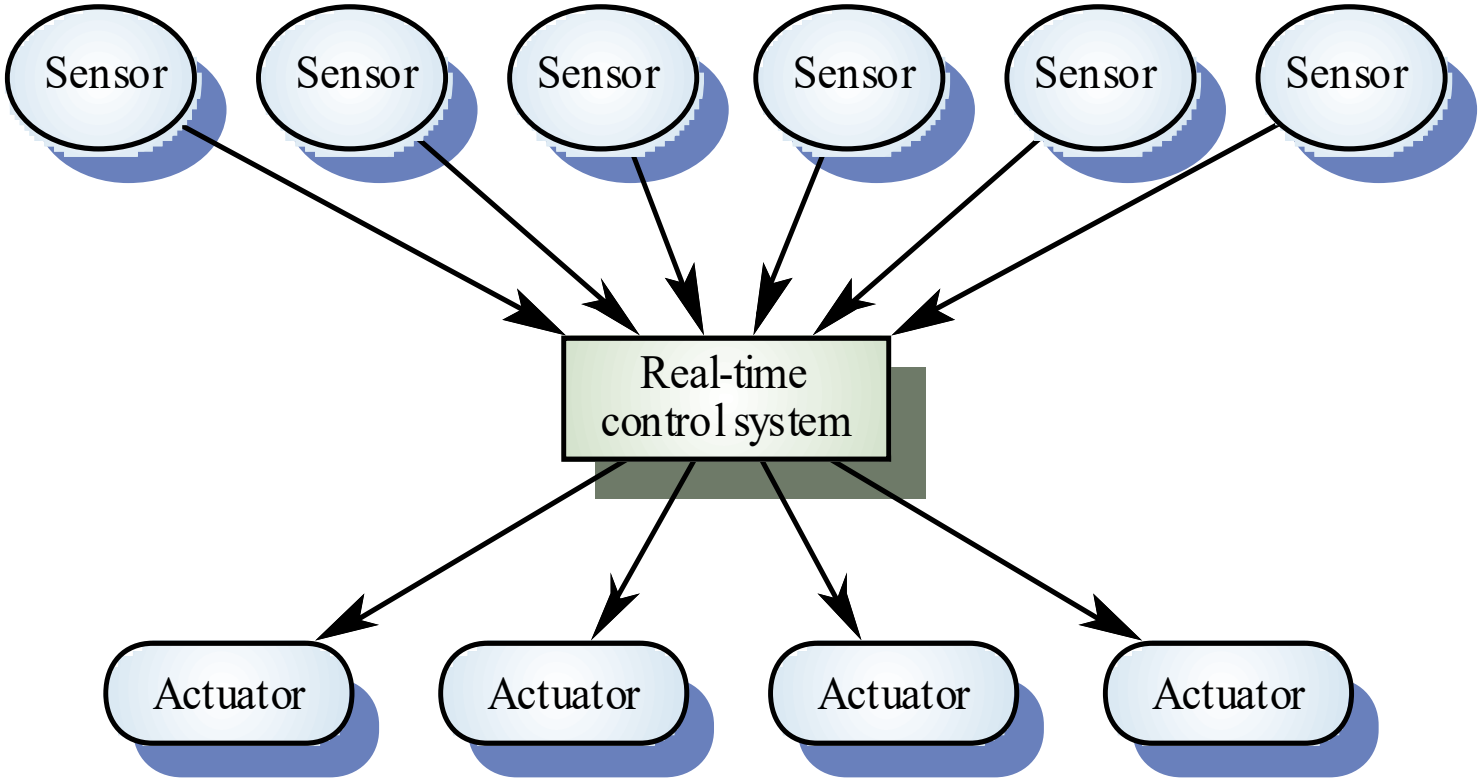


Model-View-Controller

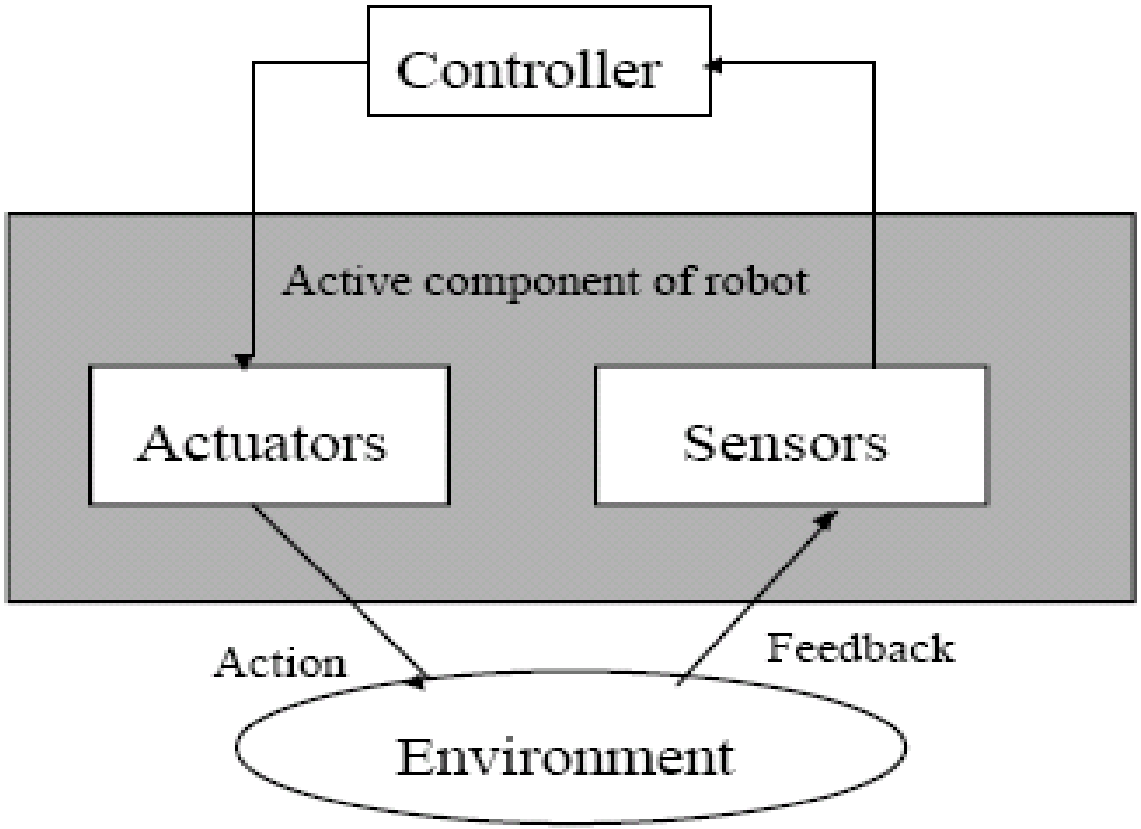
- The model consists of application data and business rules
- The controller mediates input, converting it to commands for the model or view
- A view can be any output representation of data, such as a chart or a diagram
  - Multiple views of the same data are possible, such as a pie chart for management and a tabular view for accountants
- The central idea behind MVC is code reusability and separation of concerns



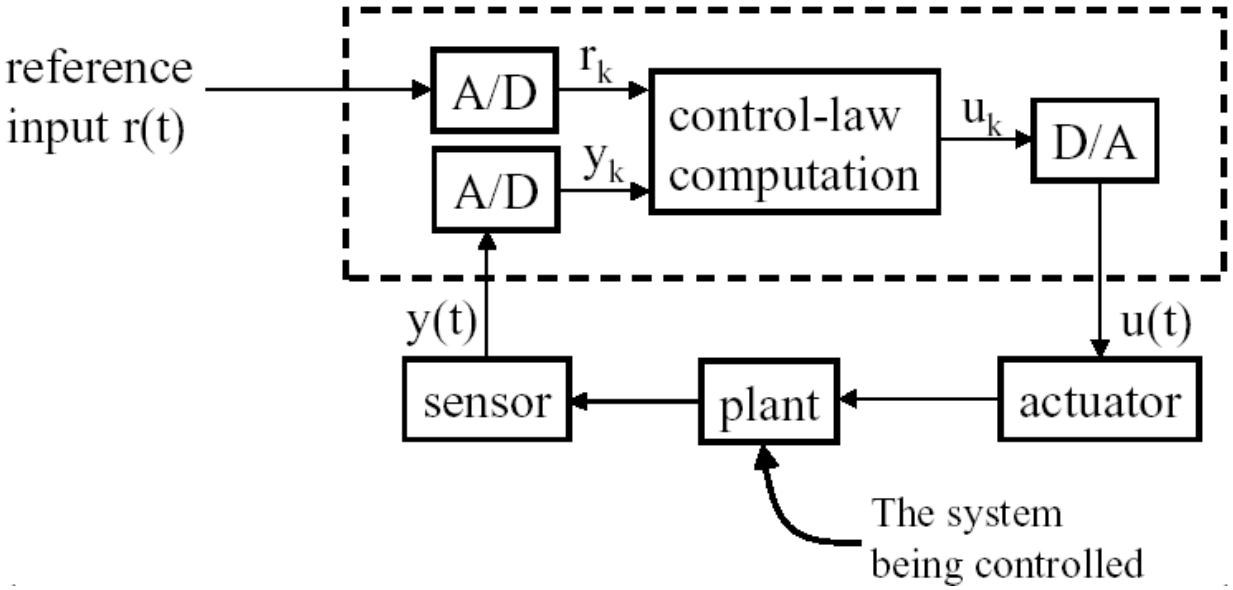
Automatic Control Systems



Control Loop



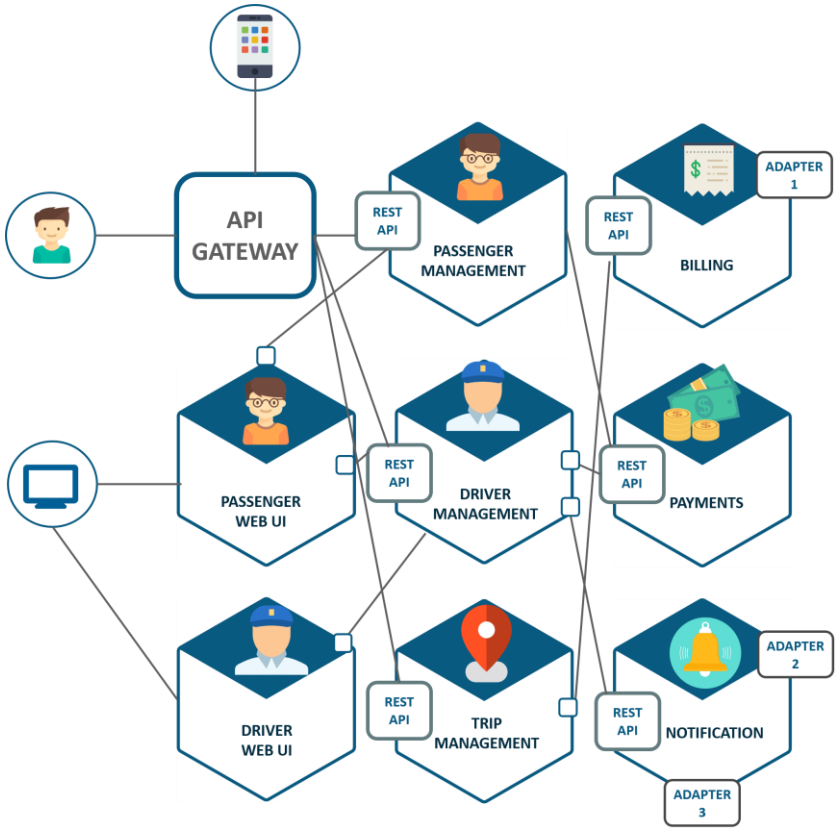
Example



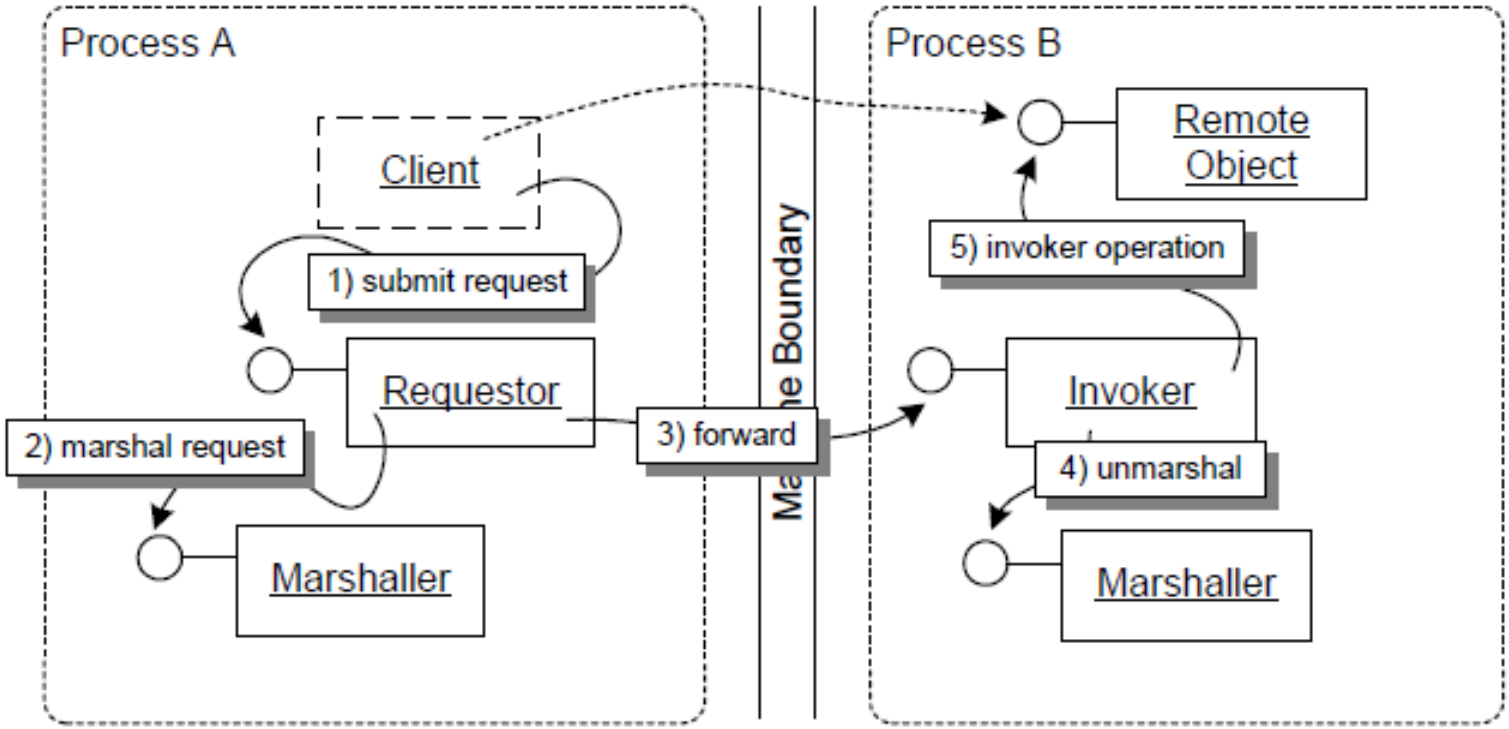

```
set timer to interrupt periodically with period T;
at each timer interrupt do
    do analog-to-digital conversion to get y;
    compute control output u;
    output u and do digital-to-analog conversion;
od
```

Microservice Architecture

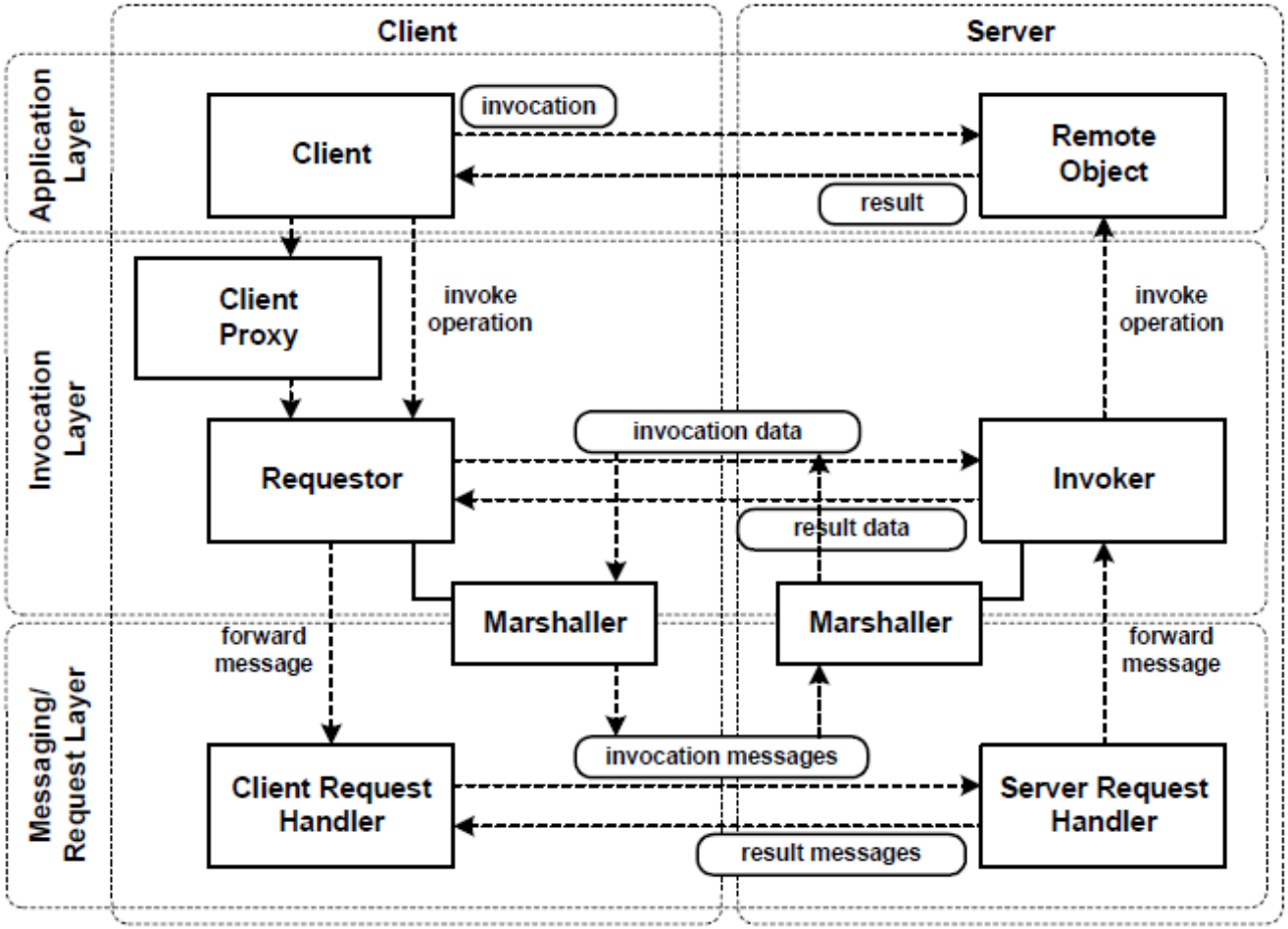
- Microservice architecture arranges an application as a collection of loosely coupled services



Remote Procedure Call

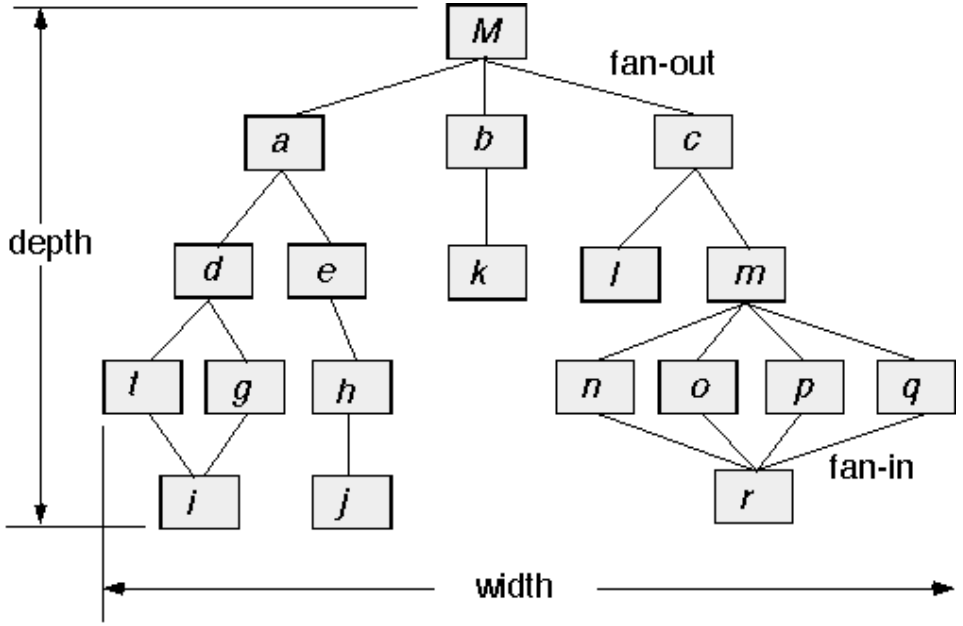


RPC + Layered


Client
Server
Application Layer
Invocation Layer
Messaging/ Request Layer
Client
invocation
Remote Object
result
Client Proxy
invoke operation
invoke operation
Requestor
invocation data
Invoker
result data
Marshaller
Marshaller
forward message
forward message
Client Request Handler
invocation messages
Server Request Handler
result messages

Call and Return Architecture

- This architecture decomposes function into a control hierarchy where a “main” program invokes a number of program components, which in turn may invoke still other components



Contents

1. Definitions
2. View Model
3. Architecture Patterns
4. Architecture Documentation

Architecture Documentation

- Diagrams
- FMC
- Architecture Description Languages

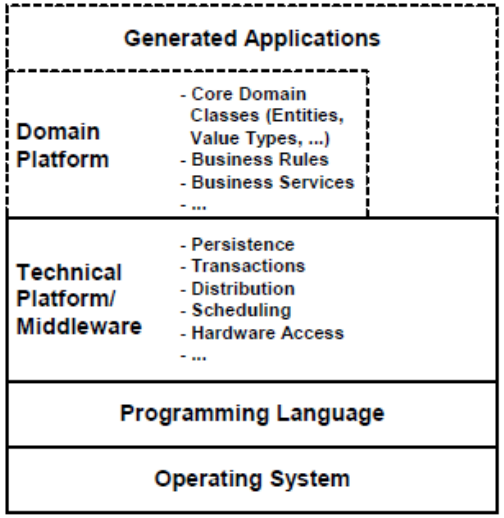
Diagrams

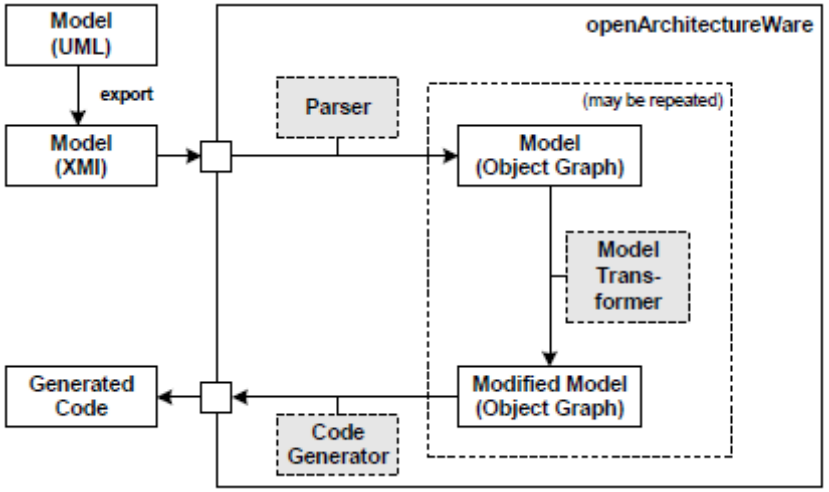
- A diagram is a simplified and structured visual representation of concepts, ideas, constructions, relations, statistical data, and anatomy used in all aspects of human activities to visualize and clarify the topic

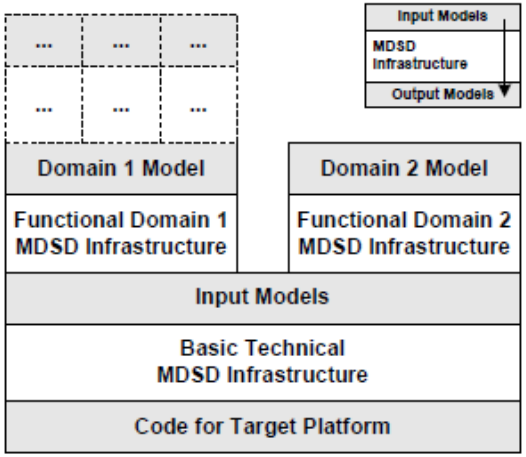
- Diagram means a graphical or symbolic representation of something, usually showing the relationship between several items

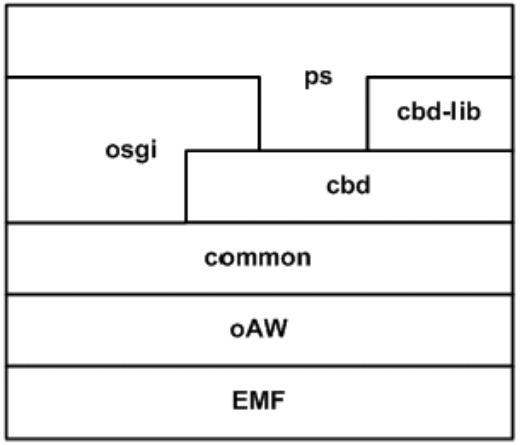
- A diagram is a drawing, sketch, plan, or chart that helps to make something easier to understand

Examples

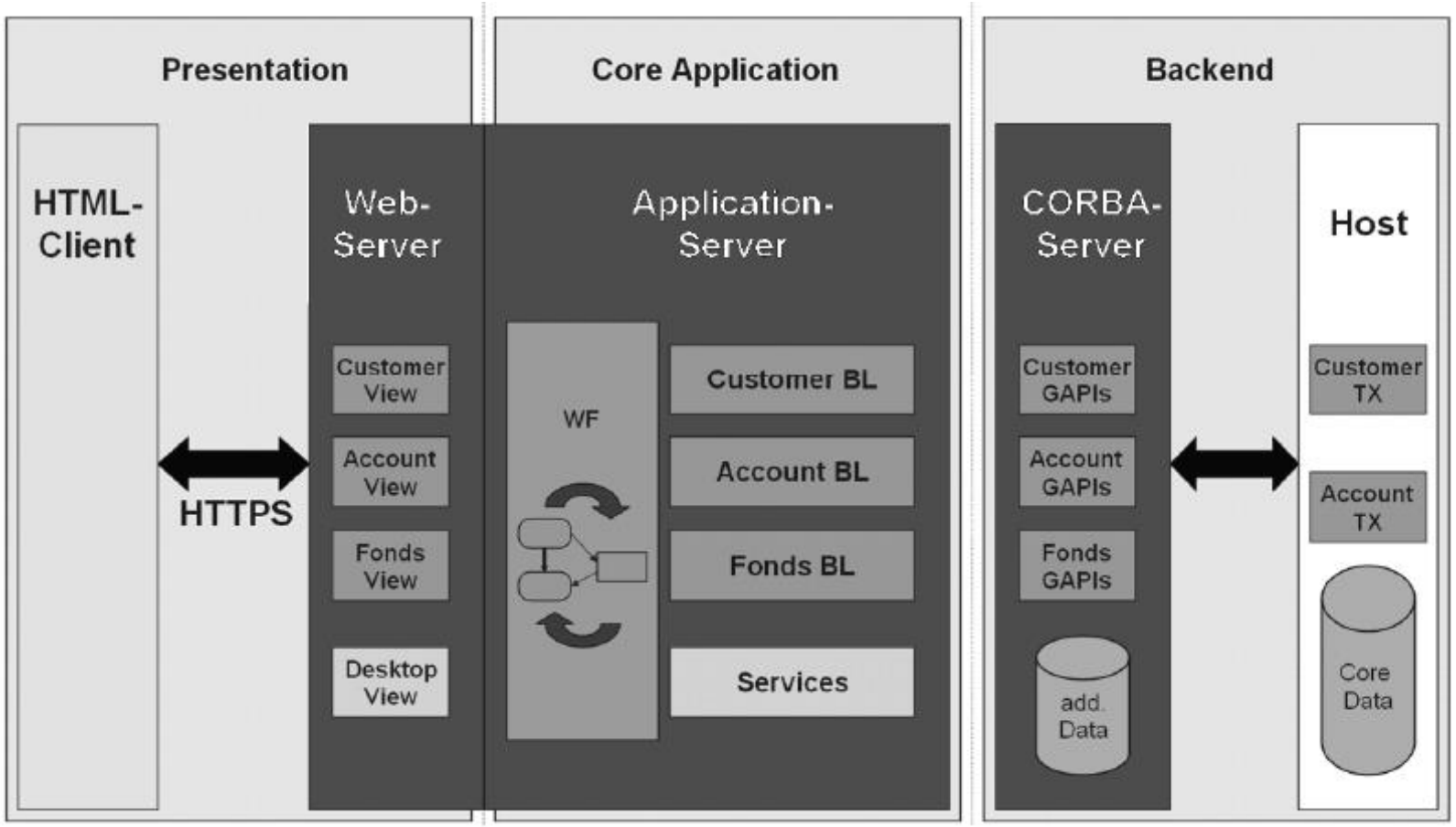

Generated Applications
Domain Platform
- Core Domain Classes (Entities, Value Types, ...)
- Business Rules
- Business Services
- ...
Technical Platform/ Middleware
- Persistence
- Transactions
- Distribution
- Scheduling
- Hardware Access
- ...
Programming Language
Operating System


Model (UML)
export
Model (XMI)
openArchitectureWare
Parser
(may be repeated)
Model (Object Graph)
Model Trans-former
Modified Model (Object Graph)
Generated Code
Code Generator

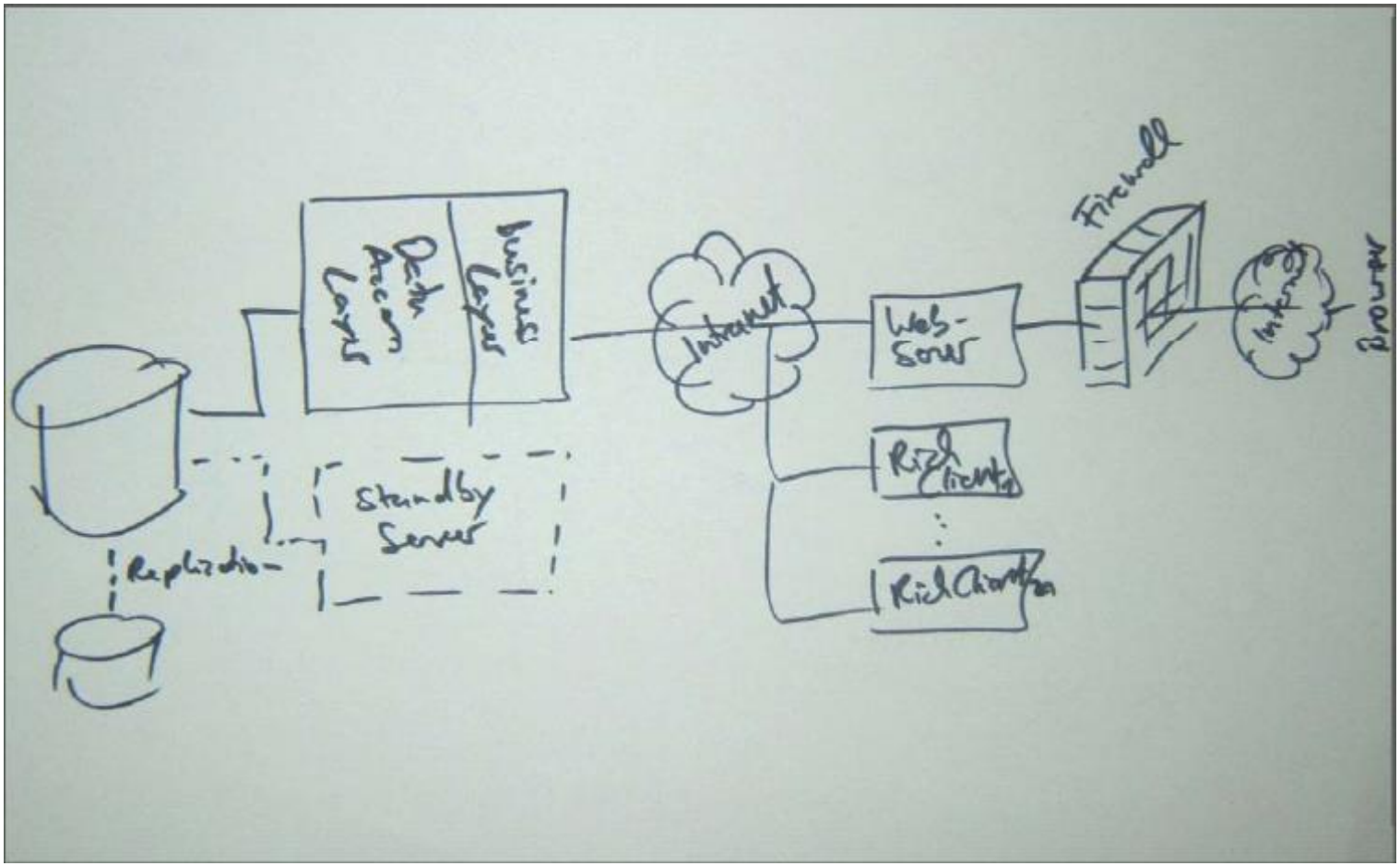

Input Models
MDSD Infrastructure
Output Models
Domain 1 Model
Domain 2 Model
Functional Domain 1 MDSD Infrastructure
Functional Domain 2 MDSD Infrastructure
Input Models
Basic Technical MDSD Infrastructure
Code for Target Platform


ps
cbd-lib
osgi
cbd
common
oAW
EMF

Example: A Three-Tier Enterprise System



No Need for a Fancy Tool



The Use of Diagrams

- They often use nice, intuitive symbols, they are (typically) not based on a well-defined (modeling) language

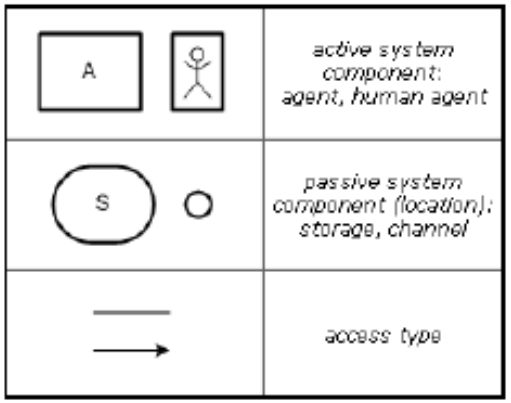
- Often, the meaning is not really clear
  - You need explaining text or somebody talking to you as they draw the diagram

- However, diagrams are very useful in documenting architectures, as long as
  - You explain what the diagram means
  - And you are consistent with respect to the notation among the set of diagrams you use

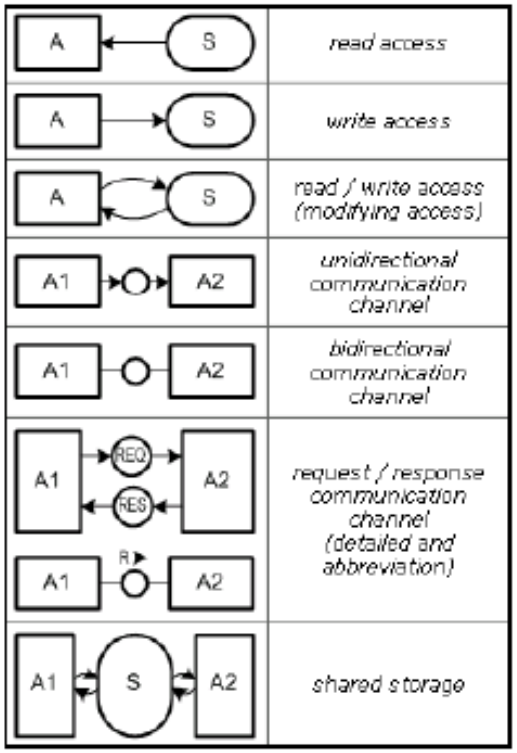
FMC (Fundamental Modeling Concepts)

- Developed by the FMC Consortium (SAP, …)

Basic Elements

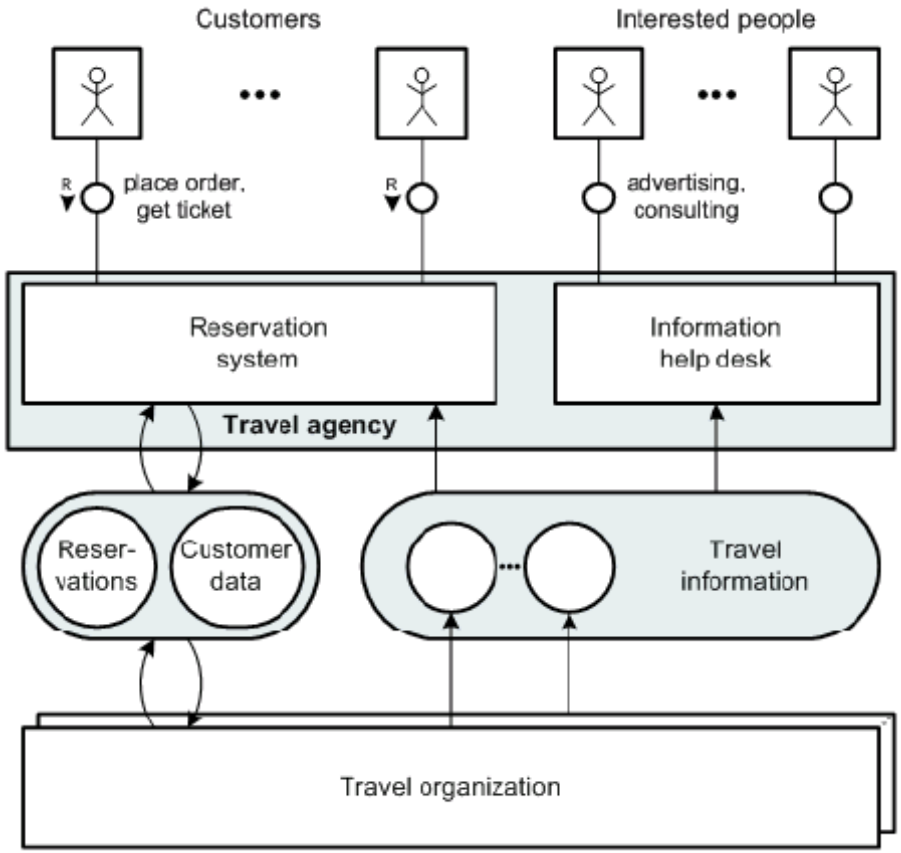


Common Structures



Example

- The Travel Organization reads and writes various data
- The travel agency's Reservation system does the same
- The Reservation system and the Info help desk only read travel information
- Customers use a request/response scheme to place orders and get tickets



Architecture Description Languages

- ADLs are predefined and formal modeling languages specifically designed to describe architectures (as opposed to software in general as in UML)

- Typically, an ADL is defined by either a university, a research department or an industry consortium for a specific domain
  - Their practical use is limited

- ADLs are mostly used in the following domains:
  - Embedded systems
  - Real-time systems
  - Safety critical systems

Architecture Description Languages

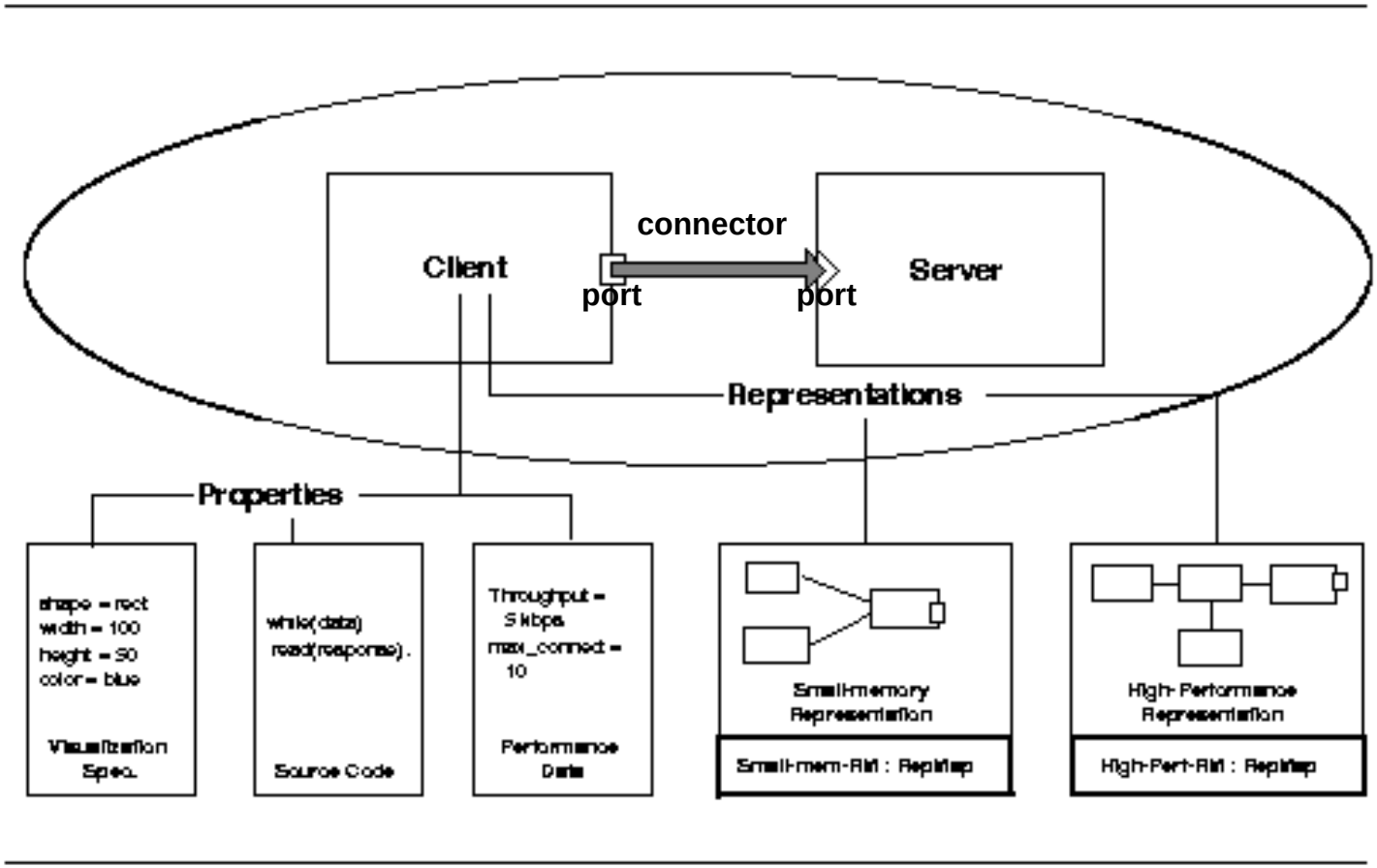
- Several different ADLs have been developed by different organizations, including;
  - Wright (developed by Carnegie Mellon)
  - Acme (developed by Carnegie Mellon)
  - xADL (developed by UCI)
  - Darwin (developed by Imperial College London)
  - DAOP-ADL (developed by University of Málaga)

Acme

- **Built on several types of architectural representation**
  - **Components represent the primary computational elements and data stores of a system**
    - Typical examples of components include such things as clients, servers, filters, objects, blackboards, and databases
  - **Connectors represent interactions among components**
    - Computationally speaking, connectors mediate the communication and coordination activities among components
    - Informally they provide the "glue" for architectural designs
    - Examples include simple forms of interaction, such as pipes, procedure call, and event broadcast
  - **Systems represent configurations of components and connectors**

Acme

- A simple client-server architecture (graphical)



Acme

❒ A simple client-server architecture (textual)

```
System simple_cs = {
  Component client = {
    Port send-request;
    Property Aesop-style : style-id = client-server;
    Property UniCon-style : style-id = client-server;
    Property source-code : external = "CODE-LIB/client.c";
  }
  Component server = {
    Port receive-request;
    Property idempotence : boolean = true;
    Property max-concurrent-clients : integer = 1;
    source-code : external = "CODE-LIB/server.c";
  }
  Connector rpc = {
    Role caller;
    Role callee;
    Property asynchronous : boolean = true;
    max-roles : integer = 2;
    protocol : Wright = " ... ";
  }
  Attachment client.send-request to rpc.caller;
  Attachment server.receive-request to rpc.callee;
}
```


Q & A

소프트웨어 아키텍쳐에 대해 …

- 아키텍쳐는 개발 전반을 지배하는 중대한 의사결정의 요체이다
  - 예시 **1)** 어떤 **OS**를 사용할 것인가**,** 어떤 **DB**를 사용할 것인가**?**
  - 예시 **2) Layered architecture**로 할 것인가**, MVC**로 할 것인가**?**
- 물론 소프트웨어 아키텍쳐에 정답은 없다
  - 대부분 경험과 직관에 의해 좌우된다
  - 아키텍쳐의 좋고 나쁨을 객관적, 정량적으로 증명하기 어렵다
- 비록 한 장짜리 그림이더라도 통찰과 혜안이 담겨야 한다
  - 잘못된 아키텍쳐는 막대한 기술적 부채를 남기기 때문이다
- 이해당사자들에게 아키텍처를 설득하는 것은 전쟁과 같다
  - 조직의 상층부로 갈수록 비즈니스 논리를 따진다
  - 비즈니스 논리는 선명하고 이성적이나**, (**역설적으로 들리지만**)** 기술 논리는 모호하고 직관적이다
  - 비즈니스 관점에서 아키텍쳐의 가치를 설명할 수 있는가**?**
    - 매출, 영업 이익, 절감되는 금액, 단축되는 기간, …